HOMEWORK 8

Kavya Saranya K V
2025115152

1. Simulate BFS and DFS traversals for the given graph.

Code:

```
#include <stdio.h>
int a[10][10], visited[10], n;
char name[10];
void dfs(int v) {
    int i;
    visited[v] = 1;
    printf("%c ", name[v]);
    for (i = 0; i < n; i++)
        if (a[v][i] && !visited[i])
            dfs(i);
}
int main() {
    int q[10], front = 0, rear = 0, i, j, s, v;
    char c;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf(" %c", &name[i]);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    scanf(" %c", &c);
    for (i = 0; i < n; i++)
        if (name[i] == c)
            s = i;
    visited[s] = 1;
    q[rear++] = s;
    printf("BFS: ");
    while (front < rear) {
        v = q[front++];
        printf("%c ", name[v]);
        for (j = 0; j < n; j++)
            if (a[v][j] && !visited[j]) {
                visited[j] = 1;
                q[rear++] = j;
            }
    }
```

```
    for (i = 0; i < n; i++)
        visited[i] = 0;
    printf("\nDFS: ");
    dfs(s);
    printf("\n");
    return 0;
}
```

Sample Input:

```
7
A B C D E F G
0 1 0 1 0 0 1
1 0 0 0 1 1 0
0 0 0 0 0 1 0
1 0 0 0 0 1 0
0 1 0 0 0 0 1
0 1 1 1 0 0 0
1 0 0 0 1 0 0
A
```

Sample Output:

```
BFS: A B D G E F C
DFS: A B E G F C D
```

2. Find out the minimum spanning tree for the given graph using Prim's and Kruskal's algorithm.

Code:

```
#include <stdio.h>
#define INF 99999
int parent[10];
int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}
int main() {
    int a[10][10], visited[10] = {0};
    int n, i, j, k, u, v, min, total = 0;
    char name[10];
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf(" %c", &name[i]);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j] == 0)
```

```
                a[i][j] = INF;
        }
    printf("Prim's MST:\n");
    visited[0] = 1;
    for (k = 1; k < n; k++) {
        min = INF;
        for (i = 0; i < n; i++)
            for (j = 0; j < n; j++)
                if (visited[i] && !visited[j] && a[i][j] < min)
{
                    min = a[i][j];
                    u = i;
                    v = j;
                }
        visited[v] = 1;
        total = total + min;
        printf("%c - %c (%d)\n", name[u], name[v], min);
    }
    printf("Total = %d\n", total);
    printf("Kruskal's MST:\n");
    total = 0;
    for (i = 0; i < n; i++)
        parent[i] = i;
    for (k = 1; k < n; k++) {
        min = INF;
        for (i = 0; i < n; i++)
            for (j = i + 1; j < n; j++)
                if (find(i) != find(j) && a[i][j] < min) {
                    min = a[i][j];
                    u = i;
                    v = j;
                }
        parent[find(u)] = find(v);
        total = total + min;
        printf("%c - %c (%d)\n", name[u], name[v], min);
    }
    printf("Total = %d\n", total);
    return 0;
}
```

Sample Input:

```
5
A B C D E
0 1 7 0 0
1 0 5 4 3
7 5 0 0 6
```

```
0 4 0 0 2
0 3 6 2 0
```

Sample Output:

```
Prim's MST:
A - B (1)
B - E (3)
E - D (2)
B - C (5)
Total = 11
Kruskal's MST:
A - B (1)
D - E (2)
B - E (3)
B - C (5)
Total = 11
```

3. Find the minimum spanning tree for the given graphs.

Code:

```
#include <stdio.h>
#define INF 99999
int parent[10];
int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}
int main() {
    int a[10][10];
    int n, i, j, k, u, v, min, total = 0;
    char name[10];
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf(" %c", &name[i]);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j] == 0)
                a[i][j] = INF;
        }
    for (i = 0; i < n; i++)
        parent[i] = i;
    for (k = 1; k < n; k++) {
        min = INF;
        for (i = 0; i < n; i++)
```

```
            for (j = i + 1; j < n; j++)
                if (find(i) != find(j) && a[i][j] < min) {
                    min = a[i][j];
                    u = i;
                    v = j;
                }
        parent[find(u)] = find(v);
        total = total + min;
        printf("%c - %c (%d)\n", name[u], name[v], min);
    }
    printf("Total = %d\n", total);
    return 0;
}
```

Sample Input 1:

```
9
1 2 3 4 5 6 7 8 9
0 10 9 6 12 0 0 0 0
10 0 0 0 8 0 0 0 0
9 0 0 7 0 5 0 0 0
6 0 7 0 8 8 7 0 0
12 8 0 8 0 0 4 0 13
0 0 5 8 0 0 14 6 0
0 0 0 7 4 14 0 8 8
0 0 0 0 0 6 8 0 10
0 0 0 0 13 0 8 10 0
```

Sample Output 1:

```
5 - 7 (4)
3 - 6 (5)
1 - 4 (6)
6 - 8 (6)
3 - 4 (7)
4 - 7 (7)
2 - 5 (8)
7 - 9 (8)
Total = 51
```

Sample Input 2:

```
8
A B C D E F G H
0 6 10 0 3 0 6 0
6 0 0 0 2 0 0 0
10 0 0 7 0 0 1 0
0 0 7 0 0 3 5 4
3 2 0 0 0 0 1 0
0 0 0 3 0 0 0 4
6 0 1 5 1 0 0 9
```

```
0 0 0 4 0 4 9 0
```
Sample Output 2:
```
C - G (1)
E - G (1)
B - E (2)
A - E (3)
D - F (3)
D - H (4)
D - G (5)
Total = 19
```
Sample Input 3:
```
8
A B C D E F G H
0 4 0 6 0 0 16 0
4 0 24 0 0 0 0 0
0 24 0 23 9 8 0 0
6 0 23 0 0 5 8 0
0 0 9 0 0 11 0 7
0 0 8 5 11 0 10 14
16 0 0 8 0 10 0 21
0 0 0 0 7 14 21 0
```
Sample Output 3:
```
A - B (4)
D - F (5)
A - D (6)
E - H (7)
C - F (8)
D - G (8)
C - E (9)
Total = 47
```

4. Dijkstra's Algorithm.

Code:
```
#include <stdio.h>
#define INF 99999
int main() {
    int a[10][10], d[10], visited[10] = {0};
    int n, s, i, j, u, min;
    char name[10], c;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf(" %c", &name[i]);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
```

```
            scanf("%d", &a[i][j]);
            if (a[i][j] == -1)
                a[i][j] = INF;
        }
    scanf(" %c", &c);
    for (i = 0; i < n; i++)
        if (name[i] == c)
            s = i;
    for (i = 0; i < n; i++)
        d[i] = a[s][i];
    d[s] = 0;
    visited[s] = 1;
    for (i = 1; i < n; i++) {
        min = INF;
        u = -1;
        for (j = 0; j < n; j++)
            if (!visited[j] && d[j] < min) {
                min = d[j];
                u = j;
            }
        if (u == -1)
            break;
        visited[u] = 1;
        for (j = 0; j < n; j++)
            if (!visited[j] && a[u][j] != INF && d[u] + a[u][j]
< d[j])
                d[j] = d[u] + a[u][j];
    }
    printf("Vertex\tDistance\n");
    for (i = 0; i < n; i++) {
        if (d[i] == INF)
            printf("%c\tUnreachable\n", name[i]);
        else
            printf("%c\t%d\n", name[i], d[i]);
    }
    return 0;
}
```

(-1 in the input matrix means there is no edge. The last line of the input is the source vertex.)

Sample Input 1:

```
7
0 1 2 3 4 5 6
0 2 6 -1 -1 -1 -1
2 0 -1 5 -1 -1 -1
6 -1 0 8 -1 -1 -1
```

```
-1 5 8 0 10 15 -1
-1 -1 -1 10 0 -1 2
-1 -1 -1 15 -1 0 6
-1 -1 -1 -1 2 6 0
0
```

Sample Output 1:

```
Vertex    Distance
0    0
1    2
2    6
3    7
4    17
5    22
6    19
```

Sample Input 2:

```
6
A B C D E F
0 4 5 -1 -1 -1
4 0 11 9 7 -1
5 11 0 -1 3 -1
-1 9 -1 0 13 2
-1 7 3 13 0 6
-1 -1 -1 2 6 0
A
```

Sample Output 2:

```
Vertex    Distance
A    0
B    4
C    5
D    13
E    8
F    14
```

Sample Input 3:

```
9
0 1 2 3 4 5 6 7 8
0 4 -1 -1 -1 -1 -1 8 -1
4 0 8 -1 -1 -1 -1 11 -1
-1 8 0 7 -1 4 -1 -1 2
-1 -1 7 0 9 14 -1 -1 -1
-1 -1 -1 9 0 10 -1 -1 -1
-1 -1 4 14 10 0 2 -1 -1
-1 -1 -1 -1 -1 2 0 1 6
8 11 -1 -1 -1 -1 1 0 7
-1 -1 2 -1 -1 -1 6 7 0
0
```

Sample Output 3:

```
Vertex    Distance
0    0
1    4
2    12
3    19
4    21
5    11
6    9
7    8
8    14
```

Sample Input 4:

```
9
A B C D E F G H I
0 5 -1 -1 -1 -1 9 18 1
5 0 7 -1 -1 -1 -1 -1 -1
-1 7 0 8 0 -1 -1 -1 6
-1 -1 8 0 20 -1 -1 -1 -1
-1 -1 0 20 0 4 -1 -1 2
-1 -1 -1 -1 4 0 1 -1 -1
9 -1 -1 -1 -1 1 0 2 3
18 -1 -1 -1 -1 -1 2 0 -1
1 -1 6 -1 2 -1 3 -1 0
A
```

Sample Output 4:

```
Vertex    Distance
A    0
B    5
C    3
D    11
E    3
F    5
G    4
H    6
I    1
```

5. Perform topological sorting for the given DAGs.

Code:

```
#include <stdio.h>
int main() {
    int a[20][20], indeg[20] = {0}, done[20] = {0};
    int n, i, j, k, count = 0;
    char name[20];
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
        scanf(" %c", &name[i]);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (a[i][j] == 1)
                indeg[j]++;
        }
    printf("Topological order: ");
    for (k = 0; k < n; k++)
        for (i = 0; i < n; i++)
            if (indeg[i] == 0 && !done[i]) {
                printf("%c ", name[i]);
                done[i] = 1;
                count++;
                for (j = 0; j < n; j++)
                    if (a[i][j] == 1)
                        indeg[j]--;
                break;
            }
    if (count < n)
        printf("\nCycle found, no topological order");
    printf("\n");
    return 0;
}
```

Sample Input 1:

```
14
m n o p q r s t u v w x y z
0 0 0 0 1 1 0 0 0 0 0 1 0 0
0 0 1 0 1 0 0 0 1 0 0 0 0 0
0 0 0 0 0 1 1 0 0 1 0 0 0 0
0 0 1 0 0 0 1 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Sample Output 1:

```
Topological order: m n p o q s r u t y v w x z
```

Sample Input 2:

```
6
1 2 3 4 5 6
0 1 1 0 0 0
0 0 0 1 1 0
0 0 0 1 0 1
0 0 0 0 1 1
0 0 0 0 0 0
0 0 0 0 0 0
```

Sample Output 2:

```
Topological order: 1 2 3 4 5 6
```

6. Insert 76, 93, 40, 47, 10 and 55 in a hash table of size 7 using linear probing method.

Code:

```
#include <stdio.h>
int main() {
    int t[20], keys[20];
    int m, n, i, pos;
    scanf("%d %d", &m, &n);
    for (i = 0; i < n; i++)
        scanf("%d", &keys[i]);
    for (i = 0; i < m; i++)
        t[i] = -1;
    for (i = 0; i < n; i++) {
        pos = keys[i] % m;
        while (t[pos] != -1)
            pos = (pos + 1) % m;
        t[pos] = keys[i];
    }
    printf("Index\tValue\n");
    for (i = 0; i < m; i++) {
        if (t[i] == -1)
            printf("%d\t-\n", i);
        else
            printf("%d\t%d\n", i, t[i]);
    }
    return 0;
}
```

(The first line of the input is the table size and the number of keys.)

Sample Input:

```
7 6
76 93 40 47 10 55
```

Sample Output:

```
Index     Value
```

```
0    47
1    55
2    93
3    10
4    -
5    40
6    76
```

7. Consider a hash table of size = 7, hash function as hash(x) = x % 7 and collision resolution strategy to be f(i) = i². Insert 22, 30 and 50.

Code:

```
#include <stdio.h>
int main() {
    int t[20], keys[20];
    int m, n, i, j, h, pos;
    scanf("%d %d", &m, &n);
    for (i = 0; i < n; i++)
        scanf("%d", &keys[i]);
    for (i = 0; i < m; i++)
        t[i] = -1;
    for (i = 0; i < n; i++) {
        h = keys[i] % m;
        pos = h;
        j = 0;
        while (t[pos] != -1) {
            j++;
            pos = (h + j * j) % m;
        }
        t[pos] = keys[i];
    }
    printf("Index\tValue\n");
    for (i = 0; i < m; i++) {
        if (t[i] == -1)
            printf("%d\t-\n", i);
        else
            printf("%d\t%d\n", i, t[i]);
    }
    return 0;
}
```

Sample Input:

```
7 3
22 30 50
```

Sample Output:

```
Index    Value
```

```
0    -
1    22
2    30
3    -
4    -
5    50
6    -
```

8. Insert the keys 27, 43, 692, 72 into hash table of size 7, where first hash function is h1(k) = k mod 7 and second hash function is h2(k) = 1 + (k mod 5) using double hashing technique.

Code:

```
#include <stdio.h>
int main() {
    int t[20], keys[20];
    int m, n, i, j, h1, h2, pos;
    scanf("%d %d", &m, &n);
    for (i = 0; i < n; i++)
        scanf("%d", &keys[i]);
    for (i = 0; i < m; i++)
        t[i] = -1;
    for (i = 0; i < n; i++) {
        h1 = keys[i] % m;
        h2 = 1 + keys[i] % 5;
        pos = h1;
        j = 0;
        while (t[pos] != -1) {
            j++;
            pos = (h1 + j * h2) % m;
        }
        t[pos] = keys[i];
    }
    printf("Index\tValue\n");
    for (i = 0; i < m; i++) {
        if (t[i] == -1)
            printf("%d\t-\n", i);
        else
            printf("%d\t%d\n", i, t[i]);
    }
    return 0;
}
```

Sample Input:

```
7 4
27 43 692 72
```

Sample Output:

```
Index     Value
0     -
1     43
2     692
3     -
4     -
5     72
6     27
```

10. Implement a hash data structure and write initialize, find and insert routines for separate chaining hash table creation.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int key;
    struct Node *next;
};
struct Node *table[10];
int m;
void initialize() {
    int i;
    for (i = 0; i < m; i++)
        table[i] = NULL;
}
void insert(int key) {
    struct Node *n = malloc(sizeof(struct Node));
    n->key = key;
    n->next = table[key % m];
    table[key % m] = n;
}
int find(int key) {
    struct Node *t = table[key % m];
    while (t != NULL) {
        if (t->key == key)
            return 1;
        t = t->next;
    }
    return 0;
}
int main() {
    int n, i, key;
    struct Node *t;
```

```
    scanf("%d %d", &m, &n);
    initialize();
    for (i = 0; i < n; i++) {
        scanf("%d", &key);
        insert(key);
    }
    for (i = 0; i < m; i++) {
        printf("%d: ", i);
        for (t = table[i]; t != NULL; t = t->next)
            printf("%d -> ", t->key);
        printf("NULL\n");
    }
    scanf("%d", &key);
    if (find(key))
        printf("%d found\n", key);
    else
        printf("%d not found\n", key);
    return 0;
}
```

(The first line of the input is the table size and the number of keys. The last line is the key to search.)

Sample Input:

```
7 7
15 11 27 8 12 22 4
27
```

Sample Output:

```
0: NULL
1: 22 -> 8 -> 15 -> NULL
2: NULL
3: NULL
4: 4 -> 11 -> NULL
5: 12 -> NULL
6: 27 -> NULL
27 found
```